

AMLGuard

Manual de Estudo - Volume 7

Demo publica, auditoria de portfolio e release engineering

Fase	Dias	Resultado
Portfolio e encerramento	21 a 24	Demo publica validada, historico auditado e release v1.0.0 preparada com evidencia reproduzivel.

Objetivo deste volume: explicar, em linguagem simples e conectada ao projeto real, como o AMLGuard transformou uma implementacao tecnicamente completa em uma experiencia acessivel para recrutadores e em uma release de portfolio versionada, auditavel e honesta.

Mensagem central

Um projeto de Machine Learning nao termina quando o modelo funciona. Para virar portfolio profissional, ele precisa ser demonstravel, verificavel, compreensivel e versionado sem exagerar o que foi realmente provado.

Dia	Entrega central	Prova principal
21	Demo Streamlit local	score, alert, threshold e model_version em paridade com o contrato congelado
22	Demo publica	aplicacao em Streamlit Community Cloud validada em Python 3.12
23	Auditoria de portfolio	22/22 dias aprovados, links e higiene do repositorio verificados
24	Release engineering	changelog, release notes, gate final, tag e release v1.0.0

modelo congelado	demo publica	auditoria	release v1.0.0
------------------	--------------	-----------	----------------

Distincao indispensavel

Release do projeto v1.0.0, contrato local do modelo 0.1.0 e Azure ML Model Asset AMLGuard:1 pertencem a dominios de versao diferentes. A release final nao retreina nem troca silenciosamente o modelo.

1. Dia 21 - da inferencia tecnica para uma demo compreensivel

A API e o endpoint Azure provavam que o sistema podia servir inferencias. Mas um recrutador nao deveria precisar configurar credenciais cloud, ler JSON bruto ou manter infraestrutura paga apenas para compreender o projeto. O Dia 21 criou uma camada de apresentacao leve em Streamlit.

O contrato que a interface preserva

Campo	Valor validado	Significado
risk_score	0.9853656292 (0.9854 na tela)	score de ranking produzido pelo artefato congelado
alert	true	decisao obtida ao comparar o score com o threshold
threshold	0.892163	operating point publico congelado
model_version	0.1.0	versao do contrato local do modelo

```
Payment Format: ACH
Amount Paid: 13701.30
sender_previous_tx_count: 238
is_business_hours: 0
same_account: 0
```

Por que congelar o artefato?

A demo usa artifacts/model.joblib com SHA-256

b9c069b8fe40f7d905b59a1af6ba2d9c21b1c848daf5c78fbbd50ad94cfe1d2e. Assim, a interface nao depende de retreinar e pode provar exatamente qual modelo gerou o resultado.

Arquitetura da demo local

formulario Streamlit	payload validado	predict_transaction	resultado exibido
----------------------	------------------	---------------------	-------------------

Analogia do mundo real - painel de demonstracao

Pense em uma fabrica com maquinas complexas. A API e como a interface industrial usada por engenheiros. A demo e como um painel de exposicao: ela nao substitui a fabrica, mas permite que uma pessoa entenda o fluxo e veja um resultado real sem operar toda a infraestrutura.

O que o Dia 21 nao afirma

- O score nao e uma probabilidade calibrada de lavagem de dinheiro.
- A interface nao transforma o projeto em sistema autonomo de compliance.
- A validacao local nao era ainda prova de disponibilidade publica.
- O endpoint Azure continuava separado da experiencia de portfolio.

2. Dia 22 - publicacao em Streamlit Community Cloud

O Dia 22 tornou a demonstracao acessivel por URL publica. A escolha do Streamlit Community Cloud separou duas necessidades: o endpoint Azure comprovou serving cloud gerenciado; a aplicacao Streamlit forneceu uma experiencia de portfolio de baixo custo.

Configuracao	Valor validado
Plataforma	Streamlit Community Cloud
Repositorio	caiobernardinelli/amlguard
Branch	main
Entrypoint	demo/app.py
Dependencias	demo/requirements.txt
Runtime	Python 3.12
Segredos necessarios	Nao
URL	https://amlguard-demo.streamlit.app

Incidente de deploy e aprendizagem

Primeira tentativa: Python 3.14.6

A primeira publicacao falhou durante a importacao da cadeia de dependencias do modelo persistido. A aplicacao foi removida e recriada com Python 3.12, alinhado ao ambiente validado localmente.

Resolucao

Fixar a versao do runtime nao foi um detalhe cosmetico. Artefatos persistidos carregam dependencias de serializacao e bibliotecas numericas; a compatibilidade do ambiente faz parte do contrato de reproducibilidade.

Como a validacao publica foi aceita

1. A URL publica abriu corretamente em navegador.
2. O formulario foi enviado com os valores sinteticos padrao.
3. A tela exibiu score 0.9854, alert true, threshold 0.892163 e model_version 0.1.0.
4. A paridade foi comparada com o contrato local congelado.

Um cliente urllib recebeu redirecionamentos HTTP 303 repetidos na camada de roteamento da plataforma. Por isso, esse cliente nao foi usado como gate de disponibilidade. O criterio aceito foi renderizacao no navegador mais submissao publica bem-sucedida.

Licao central

Um teste de disponibilidade deve medir a experiencia que realmente importa. Quando uma camada de roteamento trata clientes de forma diferente, o criterio de aceite precisa ser explicito, reproduzivel e honesto sobre suas limitacoes.

3. Tres dominios de versao - projeto, modelo e cloud

O fechamento do projeto introduz a release v1.0.0, mas isso nao deve ser confundido com a versao do modelo. Versoes diferentes respondem a perguntas diferentes.

Dominio	Identificador	Pergunta respondida
Release do projeto	v1.0.0	Qual conjunto de codigo, docs, evidencias e artefatos foi publicado?
Pacote Python	1.0.0	Qual versao instalavel do software acompanha a release?
Contrato do modelo local	0.1.0	Qual contrato de inferencia e artefato congelado a demo usa?
Azure ML Model Asset	AMLGuard:1	Qual versao do asset foi registrada no workspace Azure?

Por que nao renomear automaticamente o modelo para 1.0.0?

Porque a release final nao treinou um novo modelo. Alterar apenas o numero do contrato do modelo criaria uma falsa impressao de mudanca estatistica ou de artefato. O correto e promover o pacote e o portfolio para 1.0.0, preservando o modelo como 0.1.0.

codigo + docs	release v1.0.0	mesmo model.joblib	modelo 0.1.0
---------------	----------------	--------------------	--------------

Semantic Versioning aplicado ao portfolio

Parte	Significado geral	Aplicacao no AMLGuard
MAJOR	mudanca incompativel ou marco principal	primeira release completa e auditada do portfolio
MINOR	nova capacidade compativel	futuras funcionalidades como grafos ou calibracao
PATCH	correcao compativel	ajustes de documentacao, testes ou bugs sem novo contrato

Regra pratica

A tag deve apontar para o commit final que contem o manual, a evidencia e todos os arquivos de release. Criar a tag antes do commit final quebraria a rastreabilidade.

Autoteste 1

1. Publicar v1.0.0 significa que um novo modelo foi treinado?
2. Por que AMLGuard:1 e model_version 0.1.0 podem coexistir?

4. Dia 23 - auditoria de portfolio orientada por evidencia

O Dia 23 verificou se o repositório realmente sustenta a narrativa apresentada no README. A auditoria não se limitou a procurar arquivos recentes: ela reconstruiu a entrega dos Dias 1 a 22 usando manuais, histórico Git, código atual e evidências estruturadas.

Faixa	Fonte histórica	Comprovação atual
Dias 1-6	Manual principal, Parte 3	baseline, schema, features, treino, avaliação e predição
Dia 7	commit Git exato	24 testes iniciais e higiene Ruff
Dias 8-10	Volume 2	FastAPI, Pydantic e Docker
Dias 11-12	Volume 3	CI/CD e GHCR
Dias 13-14	Volume 4	MLflow Tracking, PyFunc e aliases
Dias 15-17	Volume 5	Azure workspace, assets e components
Dias 18-22	docs, JSON e commits	pipeline, endpoint, monitoramento e demo

Hierarquia de evidencia

Tipo	O que prova	Exemplo
Implementação atual	o artefato existe agora	src/models/predict.py
Histórico Git	a entrega ocorreu em um ponto identificável	commit Day 7
Evidência de execução	um run validado produziu resultado	day19_online_endpoint.json
Manual de estudo	o por que e o aprendizado conceitual	Volumes 3 a 7

Resultado da auditoria

22/22 dias aprovados. Links Markdown locais sem quebra. Nenhum dataset bruto, credencial ou JSONL de coleta versionado. A URL pública aparece no README e na documentação da demo.

O falso negativo do Dia 20

Uma versão intermediária do auditor procurou o texto DO NOT RETRAIN, enquanto o JSON estruturado registrava DO_NOT_RETRAIN. A correção foi alinhar o teste ao valor real da evidência. Isso demonstra por que validadores precisam testar contratos reais, não frases aproximadas.

Lição de engenharia

Auditar o portfolio é semelhante a testar software: uma narrativa bonita não substitui um contrato verificável. Cada afirmação relevante deve apontar para código, commit, evidência ou limitação documentada.

5. Como um recrutador navega pelo AMLGuard

Rota de 60 segundos

1. Abrir a demo publica e executar a inferencia sintetica.
2. Ler o TL;DR e as metricas do operating point.
3. Revisar a tabela de arquitetura de producao.
4. Abrir a auditoria de portfolio e a release final.

Rota tecnica de 5 minutos

1. Entender o problema de desbalanceamento extremo e a inadequacao da accuracy.
2. Revisar sender_previous_tx_count e a prevencao de leakage.
3. Comparar Average Precision, precision, recall e alert rate.
4. Inspeccionar os gates absolutos e de regressao.
5. Verificar contratos FastAPI, testes, Docker e CI/CD.
6. Analisar MLflow, Azure pipeline, Model Asset e serving.
7. Revisar monitoramento, PSI, limitacoes e decisao de retraining.
8. Confirmar que a demo publica usa o mesmo artefato congelado.

Palavras-chave ATS com prova

Palavra-chave	Prova no repositorio
Machine Learning / XGBoost	treino, avaliacao, baseline e artifact
Feature engineering	build_features.py e testes de paridade
MLOps / MLflow	tracking, PyFunc e model registry
FastAPI / Pydantic	health, model-info, predict e predict-batch
Docker / GHCR	Dockerfile, compose e workflow CI
CI/CD / GitHub Actions	Ruff, pytest, Hadolint e publish gate
Azure Machine Learning	Data Asset, Environment, components, pipeline e endpoint
Model monitoring / drift	Azure Monitor, Data Collector, PSI e retraining policy
Streamlit	demo publica validada

Portfolio honesto

A palavra-chave so deve aparecer no curriculo ou no README quando existe implementacao e evidencia correspondente. O AMLGuard evita declarar Kubernetes, Terraform, producao bancaria real ou SLA porque essas capacidades nao foram demonstradas.

6. Dia 24 - release engineering e gate final

Release engineering é a disciplina de transformar um estado validado do repositório em um marco identificável e reproduzível. O Dia 24 adiciona changelog, release notes, documentação final, manual de estudo, evidência e tag anotada.

Artefato	Função
CHANGELOG.md	histórico legível das capacidades entregues e controles de segurança/custo
docs/FINAL_RELEASE.md	contrato do encerramento e procedimento de release
docs/RELEASE_NOTES_v1.0.0.md	texto versionado que alimenta a GitHub Release
Volume 7 PDF	manual conceitual dos Dias 21-24
day24_release_candidate.json	evidência estruturada do gate final
tag v1.0.0	referência imutável para o commit final validado

Ordem correta da release

arquivos finais	validação	commit + push	tag + release
-----------------	-----------	---------------	---------------

1. aplicar documentos e manual
2. executar Ruff + 56 testes + validator Day 24
3. revisar diff e criar commit final
4. push da main
5. `git tag -a v1.0.0 <commit>`
6. `git push origin v1.0.0`
7. criar GitHub Release usando as release notes

Release candidate versus release publicada

Estado	Característica
Release candidate	arquivos e gates prontos; tag ainda disponível e não criada
Release publicada	commit final enviado, tag v1.0.0 criada e GitHub Release registrada

Controle de custo preservado

A release não recria o Managed Online Endpoint. O código e as evidências comprovam o deploy Azure; a experiência pública permanece no Streamlit Community Cloud.

7. O que o validador final precisa verificar

Gate	Critério de aceite
Versoes	pacote 1.0.0; modelo local continua 0.1.0; Azure asset documentado como AMLGuard:1
Modelo	SHA-256 do artifacts/model.joblib permanece inalterado
Qualidade	Ruff limpo e exatamente 56 testes aprovados
Historico	evidencia Day 23 PASS com 22 dias aprovados
Evidencias	Days 18-23 continuam com status PASS
Documentacao	README, changelog, release notes e manual presentes
Links	nenhum link Markdown local quebrado
Higiene	sem dataset bruto, credenciais, chaves ou JSONL de coleta
Git	branch main, tag v1.0.0 ainda disponivel antes do commit/tag

Por que executar novamente testes ja aprovados?

Porque a release inclui mudancas de documentacao, versao e scripts de auditoria. Mesmo alteracoes aparentemente nao funcionais podem quebrar instalacao, imports, links ou contratos de automacao. O gate final testa o conjunto que sera marcado pela tag, nao apenas o estado de dias anteriores.

Diferenca entre evidencia e garantia absoluta

- 56 testes aprovados demonstram os comportamentos cobertos pela suite, nao ausencia de qualquer bug possivel.
- Uma inferencia publica bem-sucedida demonstra o contrato validado, nao SLA de producao.
- PSI significativo no batch sintetico demonstra deteccao de mudanca, nao drift real de clientes.
- A tag torna o estado rastreavel, mas nao transforma dados sinteticos em evidencia regulatoria real.

Licao central

Release profissional e uma afirmacao delimitada: este conjunto de arquivos passou por estes gates, produz estas evidencias e possui estas limitacoes conhecidas.

Autoteste 2

1. Por que o SHA-256 do modelo e um gate da release?
2. Por que a tag precisa ser criada depois do commit final?
3. Uma GitHub Release substitui os testes?

8. Definicao de pronto dos Dias 21 a 24

Evidencia	Status esperado
Demo local reproduz score/alert/threshold/version	Concluido
Modelo congelado versionado com hash conhecido	Concluido
Demo publica abre e aceita submissao	Concluido
Runtime publico fixado em Python 3.12	Concluido
Endpoint Azure permanece desnecessario para a demo	Concluido
Dias 1-22 auditados contra manuais e repositorio	Concluido
Links Markdown locais verificados	Concluido
Higiene de arquivos versionados verificada	Concluido
README com rota para recrutadores	Concluido
Package version promovida para 1.0.0	Dia 24
Changelog e release notes versionados	Dia 24
Volume 7 presente no repositorio	Dia 24
Validador final gera status PASS	Dia 24
Commit final enviado para main	Dia 24
Tag anotada v1.0.0 enviada	Dia 24
GitHub Release criada	Dia 24

Arquivos centrais desta fase

```
demo/app.py
demo/requirements.txt
docs/RECRUITER_DEMO.md
docs/PORTFOLIO_READINESS.md
docs/FINAL_RELEASE.md
docs/RELEASE_NOTES_v1.0.0.md
docs/evidence/day21_recruiter_demo.json
docs/evidence/day22_public_deployment.json
docs/evidence/day23_portfolio_readiness.json
docs/evidence/day24_release_candidate.json
scripts/audit/validate_day23_portfolio_readiness.py
scripts/audit/validate_day24_release_candidate.py
```

Seguranca e privacidade

A demo utiliza apenas entradas sinteticas ou nao sensiveis. Credenciais Azure, dataset bruto e colecoes JSONL temporarias nao devem ser publicados.

9. Resumo para entrevista

Resposta de aproximadamente 90 segundos

Eu desenvolvi o AMLGuard, um projeto end-to-end de Machine Learning para detecção de transações suspeitas sob desbalanceamento extremo. Comecei com um notebook acadêmico e refatorei o fluxo em módulos testados para dados, features, treino, avaliação e inferência. O modelo campeão foi um XGBoost ponderado, avaliado por Average Precision e por recall em um operating point de 2% de precision, porque accuracy seria enganosa. Depois servi o contrato com FastAPI e Pydantic, containerizei com Docker, automatizei testes e publicação no GitHub Actions/GHCR, adicionei MLflow e levei o pipeline ao Azure ML com components, gates, model asset, endpoint e monitoramento. Para fechar o portfolio, publiquei uma demo Streamlit, auditei toda a história contra commits e evidências e preparei a release v1.0.0 sem esconder limitações: os dados são sintéticos, o score não é probabilidade calibrada e as decisões exigem revisão humana.

Perguntas que o projeto permite responder

- Por que accuracy falha em AML extremamente desbalanceado?
- Como impedir leakage em features históricas?
- Como transformar precision/recall em carga de trabalho para analistas?
- Como impedir promoção de um modelo degradado?
- Qual a diferença entre observabilidade do serviço e monitoramento do modelo?
- Como demonstrar cloud sem manter infraestrutura cara ligada?
- Como provar que um portfolio realmente implementou as tecnologias declaradas?

Respostas dos autotestes

Autoteste 1. (1) Não. v1.0.0 identifica a release completa de software e portfolio; o modelo congelado continua 0.1.0. (2) Porque Azure ML Model Asset e contrato local pertencem a registries e domínios diferentes.

Autoteste 2. (1) O hash prova que a release usa exatamente o mesmo artefato validado. (2) A tag deve apontar para o commit que já contém todos os arquivos finais. (3) Não. A release registra um estado; testes e validadores fornecem a evidência de qualidade desse estado.

Conclusão

O valor final do AMLGuard não está apenas no modelo. Ele está na cadeia completa de raciocínio e engenharia: problema operacional, métricas adequadas, prevenção de leakage, reproducibilidade, testes, serving, MLOps, cloud, monitoramento, demonstração pública, auditoria e release. Essa cadeia permite que outra pessoa entenda o que foi feito, reproduza partes importantes e saiba exatamente quais afirmações o projeto sustenta.

Marco final

Com a tag v1.0.0 e a GitHub Release apontando para o commit final validado, o AMLGuard deixa de ser apenas um projeto em desenvolvimento e passa a ter uma versão pública, rastreável e apresentável como portfolio profissional.